Zellic

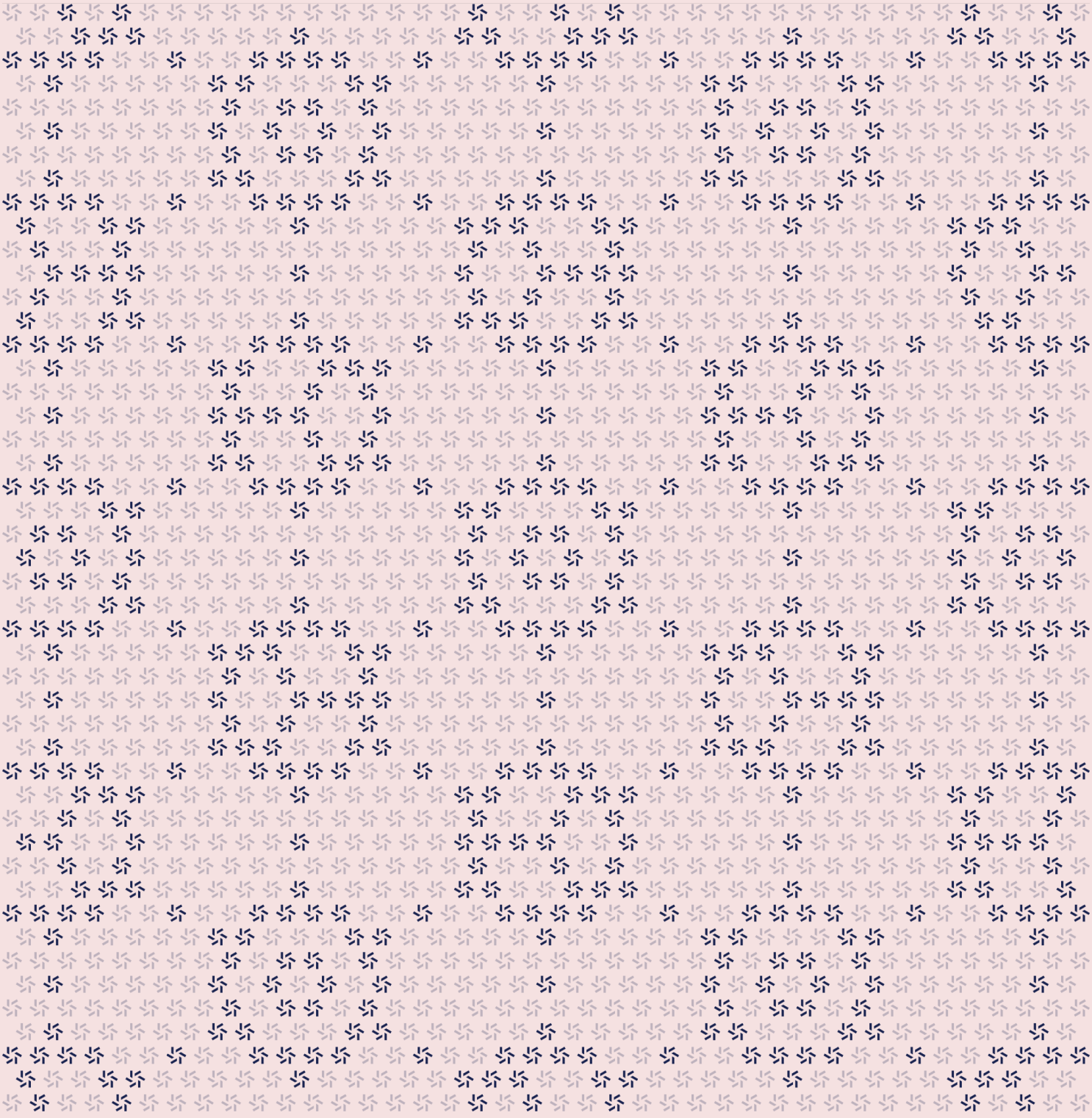
Prepared for
Kelly Zhang
LayerZero

Prepared by
Katerina Belotskaia
Sihoon Lee
Ayaz Mammadov
Zellic

October 1, 2026

Stellar OFT & Sac Manager Diff

Stellar Application Security Assessment



Contents

About Zellic **3**

1. Overview **4**

1.1. Executive Summary 4

1.2. Goals of the Assessment 4

1.3. Non-goals and Limitations 4

1.4. Results 4

2. Introduction **6**

2.1. About Stellar OFT & Sac Manager Diff 6

2.2. Methodology 6

2.3. Scope 8

2.4. Project Overview 8

2.5. Project Timeline 9

3. Detailed Findings **10**

3.1. Authorization model documentation omits `UPGRADER_ROLE` 10

3.2. Insufficient test coverage of the upgrade path 12

4. Assessment Results **15**

4.1. Disclaimer 15

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for LayerZero on September 29, 2026. During this engagement, Zellic reviewed Stellar OFT & Sac Manager Diff's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the upgrade functionality properly access controlled?
- Does making the contracts upgradeable introduce any new security risks?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

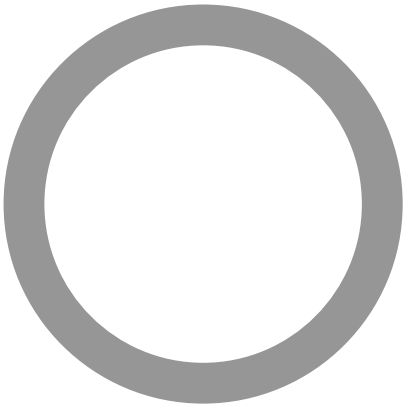
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Stellar OFT & Sac Manager Diff files, we discovered two findings, both of which were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	2



2. Introduction

2.1. About Stellar OFT & Sac Manager Diff

LayerZero contributed the following description of Stellar OFT & Sac Manager Diff:

> LayerZero works with asset issuers, chains, and applications to make money more extensible. Issue any asset on chain, extend its reach to new ecosystems, connect fragmented liquidity and more.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the files.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Stellar OFT & Sac Manager Diff Files

Type	Rust
Platform	Soroban
Target	Only changes between d20f6fa2...acf921bf
Repository	https://github.com/LayerZero-Labs/monorepo-external ↗
Version	`acf921bf7ea3e453200588b43d9dc0b414a2fe9a`
Programs	`oft/src/oft.rs` `sac-manager/src/sac_manager.rs`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 0.2 person-weeks. The assessment was conducted by three consultants over the course of 1 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io

The following consultants were engaged to conduct the assessment:

Katerina Belotskaia
Engineer
kate@zellic.io

Sihoon Lee
Engineer
sihoon@zellic.io

Ayaz Mammadov
Engineer
ayaz@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 29, 2026	Start of primary review period
September 29, 2026	End of primary review period

3. Detailed Findings

3.1. Authorization model documentation omits `UPGRADER_ROLE`

Target	sac-manager	**ID**	ZELLIC-35EW65
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The "Authorization Model" section in the module documentation of sac_manager.rs lists only `ADMIN_MANAGER_ROLE`, `MINTER_ROLE`, `BLACKLISTER_ROLE`, and `CLAWBACK_ROLE`:

```
// apps/oft-app/contracts/stellar/sac-manager/src/sac_manager.rs#L15-L22
//! ## Authorization Model
//!
//! - **Owner** (via Ownable): The deployer/admin address. Can manage TTL and
    grant/revoke roles.
//! - **RBAC roles**: Access is enforced by role.
//!   - `ADMIN_MANAGER_ROLE`: set_admin
//!   - `MINTER_ROLE`: mint (e.g. for OFT credit)
//!   - `BLACKLISTER_ROLE`: set_authorized
//!   - `CLAWBACK_ROLE`: clawback
```

However, the contract is declared with `upgradeable(no_migration, rbac)`, which exposes `upgrade` and `migrate` gated by `UPGRADER_ROLE`. This is the most powerful role, since replacing the contract code bypasses every other role check. The "Trust Model Requirement" section also does not mention that the contract is upgradable.

Impact

Readers of the documentation may underestimate the trust placed in `UPGRADER_ROLE` holders.

Recommendations

Add `UPGRADER_ROLE` (`upgrade`, `migrate`) to the "Authorization Model" section, and note in the "Trust Model Requirement" section that the contract is upgradable.

Remediation

LayerZero has acknowledged this issue and provided the following comment:

The documentation will not be updated at this time. We may address this in the future.

3.2. Insufficient test coverage of the upgrade path

Target	oft, sac-manager	**ID**	ZELLIC-2J3GTE
Category	Protocol Risks	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

OFT and SACManager are declared with `upgradeable(no_migration, rbac)`, which exposes `upgrade` and `migrate` gated by `UPGRADER_ROLE`. They are the only production contracts that use the `rbac` variant of the upgradable macro. The accompanying tests in oft/src/tests/upgrade.rs and sac-manager/src/tests/upgrade.rs cover two scenarios: an operator without the role is rejected, and an authorized `upgrade_and_migrate` succeeds.

Access control is tested only through Upgrader.

The negative test performs a single `upgrade_and_migrate` call through Upgrader:

```
// apps/oft-app/contracts/stellar/oft/src/tests/upgrade.rs#L30-L47
#[test]
fn unauthorized_operator_cannot_upgrade_through_upgrader() {
    // [...]

    let result = UpgraderClient::new(&env, &upgrader).try_upgrade_and_migrate(
        &contract,
        &wasm_hash,
        &42_u32.to_xdr(&env),
        &Some(operator),
    );

    assert_eq!(result.err().unwrap().ok().unwrap(),
    RbacError::Unauthorized.into());
}
```

Upgrader calls `upgrade` and then `migrate` on the target:

```
// contracts/protocol/stellar/contracts/upgrader/src/lib.rs#L92-L97
        if let Some(operator) = operator {
            operator.require_auth();
            let client = UpgradeableRbacClient::new(env, contract_address);
            client.upgrade(wasm_hash, operator);
```

```
            client.migrate(migration_data, operator);
        } else {
            // [...]
```

Since only the outcome of the whole call is checked, the test cannot tell which function rejected the caller.

These are other untested behaviors:

- **Signature requirement.** All tests use `mock_all_auths()`, so the `operator.require_auth()` inserted by `#[only_role]` is never exercised.
- **Owner without the role.** No test checks that the owner cannot call `upgrade` or `migrate` without holding `UPGRADER_ROLE`.
- **Other negative cases.** Rejection of a revoked operator, `migrate` without a preceding `upgrade`, and invalid migration data are not tested for the `rbac` variant.
- **Nonatomic upgrades.** Only the atomic path through Upgrader is tested; `upgrade` and `migrate` are separate entry points and can be called in separate transactions.
- **Baseline checks.** The success test does not check the state before the upgrade, for example that the new code is not yet active or that the `Migrating` flag is cleared afterwards.

Impact

Regressions in `upgrade` and `migrate` access control, such as a removed role check or a missing signature requirement, would not be detected by the current tests and could reach production unnoticed.

Recommendations

We recommend extending the upgrade tests as follows.

1. Test the access control of `upgrade` and `migrate` separately:
    - For `upgrade`, calls from an address without the role, from the owner without `UPGRADER_ROLE`, and from a revoked operator are rejected with `RbacError::Unauthorized`.
    - For `migrate`, while a migration is pending (`Migrating` is set), calls from an address without the role and from the owner without `UPGRADER_ROLE` are rejected with `RbacError::Unauthorized`.
    - For both functions, an operator with the role but without a signature is rejected, and the same call succeeds once authorized by the operator.
2. Test the contracts' own `migrate`. Without a preceding `upgrade`, it is rejected with `UpgradeableError::MigrationNotAllowed`. With `Migrating` set, non-unit migration data is rejected with `UpgradeableError::InvalidMigrationData`.

3. In the success test, assert that the new code is not active before the upgrade and that `Migrating` is cleared afterwards.

Remediation

This issue has been acknowledged by LayerZero.

4. Assessment Results

During our assessment on the scoped Stellar OFT & Sac Manager Diff files, we discovered two findings, both of which were informational in nature.

4.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.